

DCC192

2025/2



Desenvolvimento de Jogos Digitais

A7: Física I – Objetos Rígidos

Prof. Lucas N. Ferreira

Plano de Aula



- ▶ Física em Jogos Digitais
- ▶ Movimentação de Objetos Rígidos
 - ▶ Método de Euler Semi-Implicito
- ▶ Aceleração da gravidade
- ▶ Atrito
- ▶ Resistência do Meio

Física em Jogos Digitais



Geralmente, em jogos digitais queremos **mover objetos rígidos** por meio de aplicação de **forças** causadas pelo jogador ou por outros objetos do jogo:



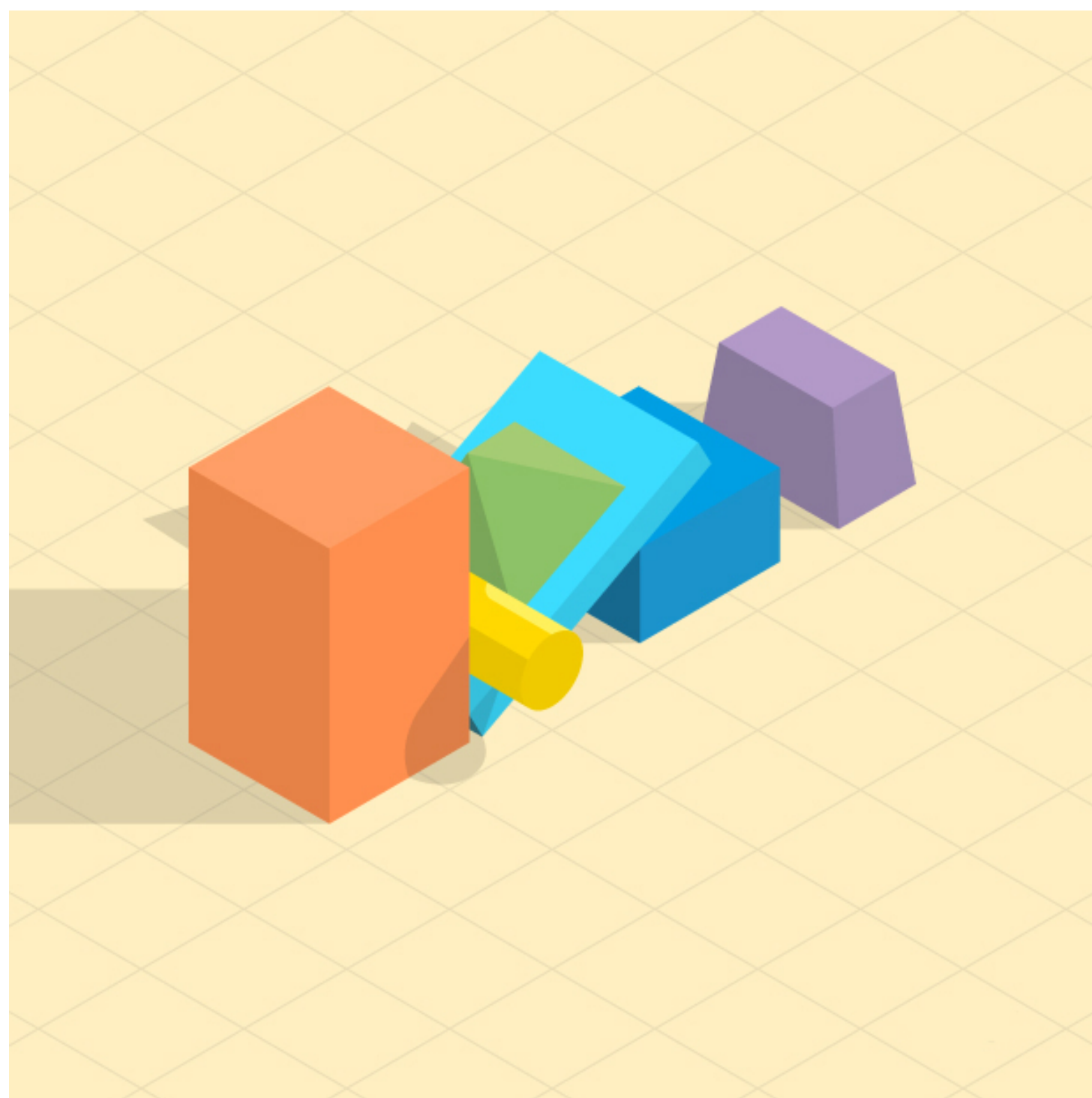
Por exemplo:

- ▶ Andar e Correr
- ▶ Pular
- ▶ Planar (resistência do ar menor)
- ▶ Deslizar (em morros inclinados)
- ▶ Vento (atuando em uma direção)
- ▶ Atrito diferente quando andando no casco
- ▶ ...

Objetos Rígidos



Geralmente, os jogos que possuem física, assumem que os *game objects* são **Objetos Rígidos**, ou seja, são sólidos que não sofrem deformação. Suas propriedades são:



- ▶ **Massa** (escalar): quantidade de matéria no corpo
- ▶ **Posição** (vetor): localização no espaço (2D ou 3D)
- ▶ **Velocidade** (vetor): taxa de variação de posição
- ▶ **Acceleração** (vetor): taxa de variação de velocidade

Apesar de objetos rígidos não existirem na vida real, são excelentes simplificações para simulações de objetos em jogos.

The diagram shows a sequence of four blue squares arranged in a zig-zag pattern, representing a path or a series of steps. Each square has a red arrow pointing downwards, representing a force component $\vec{p}$. A blue arrow pointing to the right represents a force component $\vec{r}$. A green arrow pointing upwards represents a force component $\vec{j}$. The total force vector $\vec{f}$ is shown as a green arrow pointing from the bottom-left square to the top-right square. The equation $\vec{f} = \vec{r} + \vec{j} + \vec{p}$ is written in green text at the top left. The diagram illustrates how the total force $\vec{f}$ is decomposed into its components $\vec{r}$, $\vec{j}$, and $\vec{p}$.

Força ($\vec{f}$) é igual a massa m vezes a aceleração $\vec{a}$

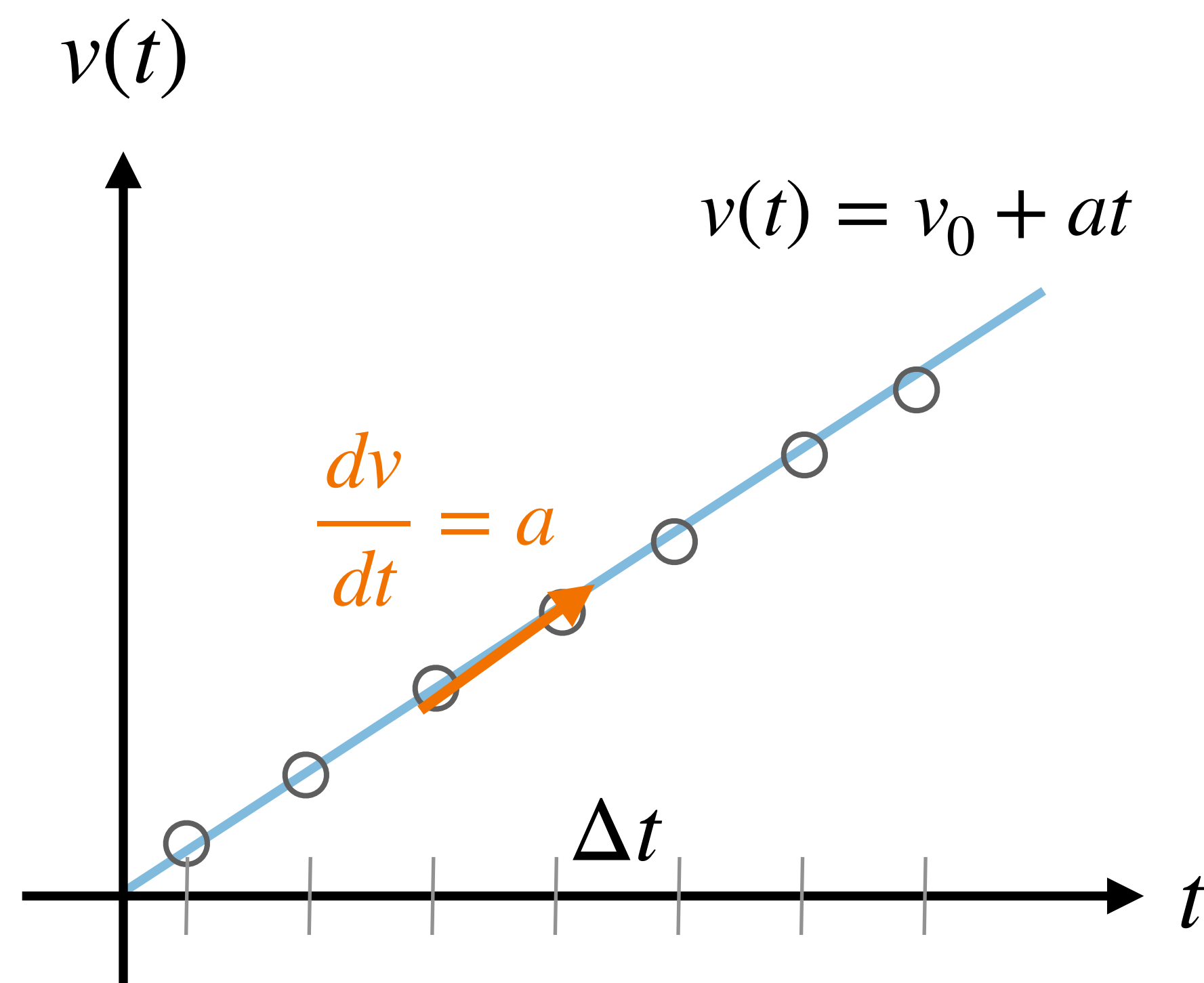
$$\vec{f} = m\vec{a} \longrightarrow \vec{a} = \frac{\vec{f}}{m} \longrightarrow \vec{a} = \frac{\sum \vec{f}_i}{m}$$

1. Como calcular a velocidade $\vec{v}$?
2. Como calcular a posição $\vec{p}$?

Movimentação de Objetos Rígidos



Precisamos aproximar a função velocidade $v(t)$ e de posição $x(t)$ em instantes discretos de tempo (quadros do jogo), separados por um intervalo de tempo Δt (*delta time*):



Objetos em jogos possuem aceleração dinâmica, mas a visualização do caso MRUV é mais simples

Derivada aproximada considerando dois quadros consecutivos t e $t + \Delta t$:

$$\frac{dv}{dt} \approx \frac{v(t + \Delta t) - v(t)}{\Delta t}$$

Isolando $v(t + \Delta t)$:

$$v(t + \Delta t) \approx v(t) + \frac{dv}{dt} \cdot \Delta t$$

Substituindo $\frac{dv}{dt} = a$:

$$v(t + \Delta t) \approx v(t) + a \cdot \Delta t$$

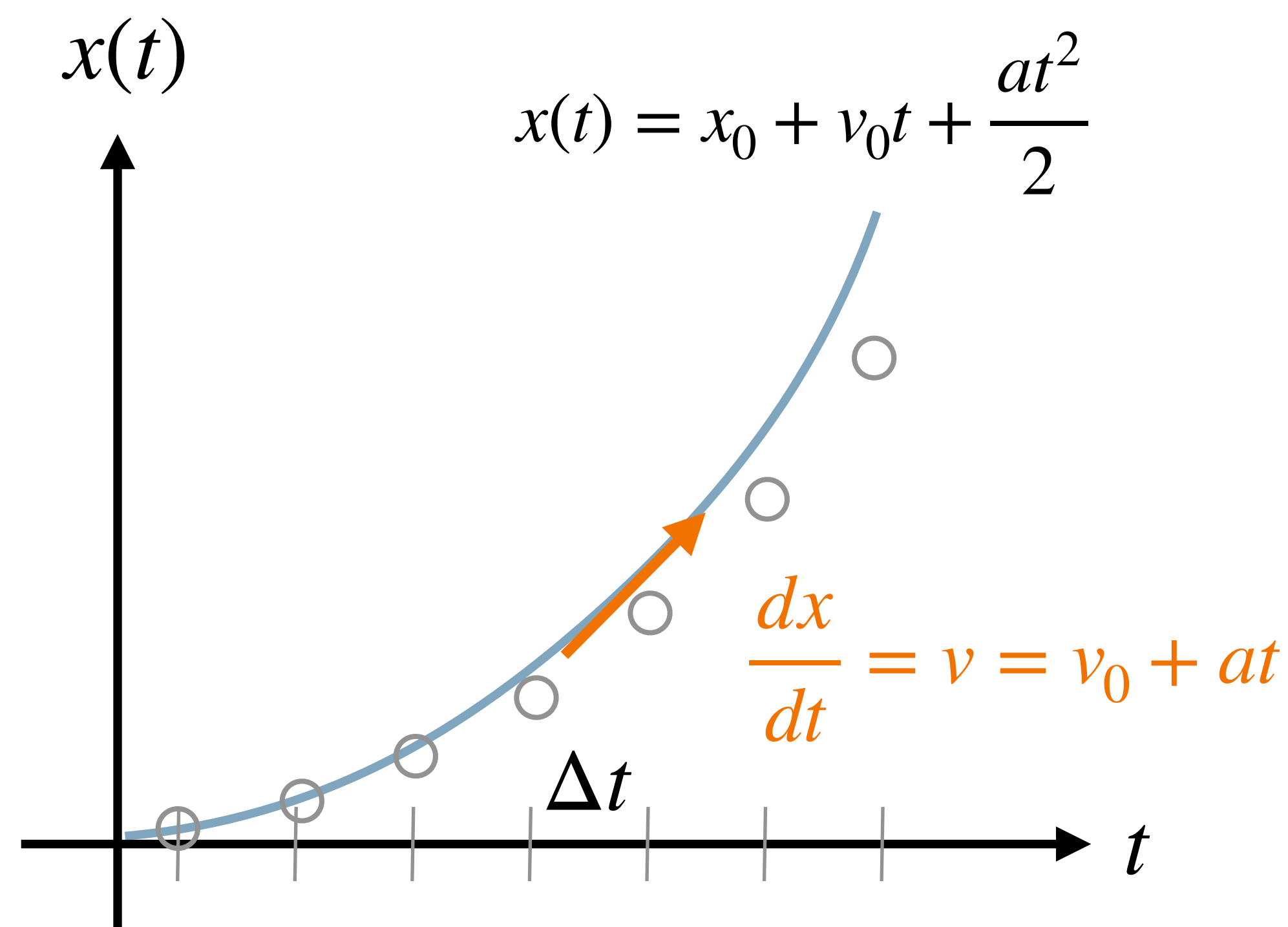
Renomeando as variáveis:

$$v' \leftarrow v + a \cdot \Delta t$$

Movimentação de Objetos Rígidos



Precisamos aproximar a função velocidade $v(t)$ e de posição $x(t)$ em instantes discretos de tempo (quadros do jogo), separados por um intervalo de tempo Δt (*delta time*):



Objetos em jogos possuem aceleração dinâmica, mas a visualização do caso MRUV é mais simples

Derivada aproximada considerando dois quadros consecutivos t e $t + \Delta t$:

$$\frac{dx}{dt} \approx \frac{x(t + \Delta t) - x(t)}{\Delta t}$$

Isolando $v(t + \Delta t)$:

$$x(t + \Delta t) \approx x(t) + \frac{dx}{dt} \cdot \Delta t$$

Substituindo $\frac{dx}{dt} = v$:

$$x(t + \Delta t) \approx x(t) + v \cdot \Delta t$$

Renomeando as variáveis:

$$x' \leftarrow x + v \cdot \Delta t$$

Esse método de aproximação é conhecido como **Método de Euler!**

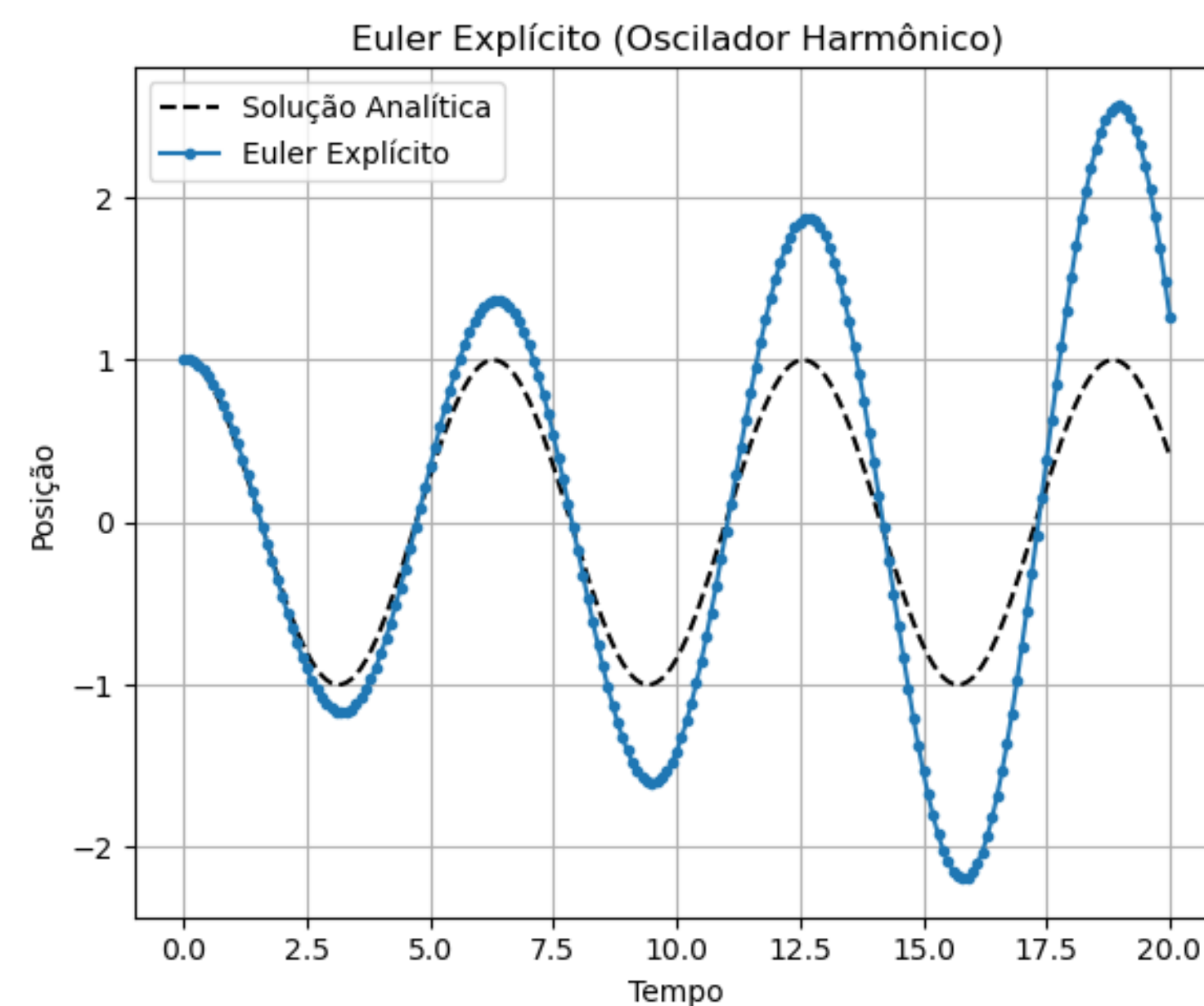
Método de Euler

Existem duas variações básicas do **Método de Euler**:



► Método de Euler Explícito

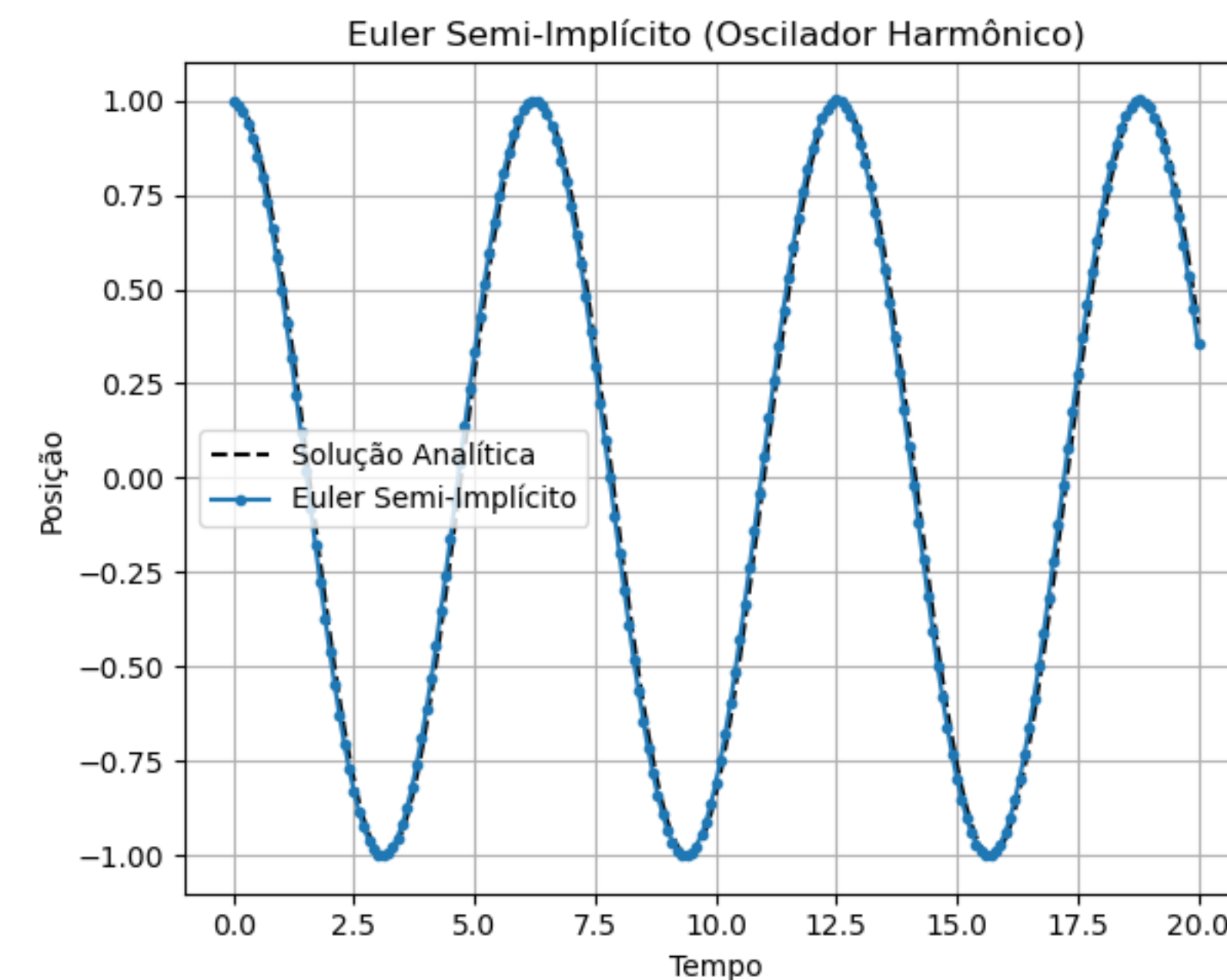
1. $\vec{p}' = \vec{p} + \vec{v}\Delta t$ Primeiro atualiza posição,
2. $\vec{v}' = \vec{v} + \vec{a}\Delta t$ depois velocidade



O sistema ganha energia ao longo do tempo!

► Método de Euler Semi-Implicito

1. $\vec{v}' = \vec{v} + \vec{a}\Delta t$ Primeiro atualiza velocidade,
2. $\vec{p}' = \vec{p} + \vec{v}'\Delta t$ depois posição

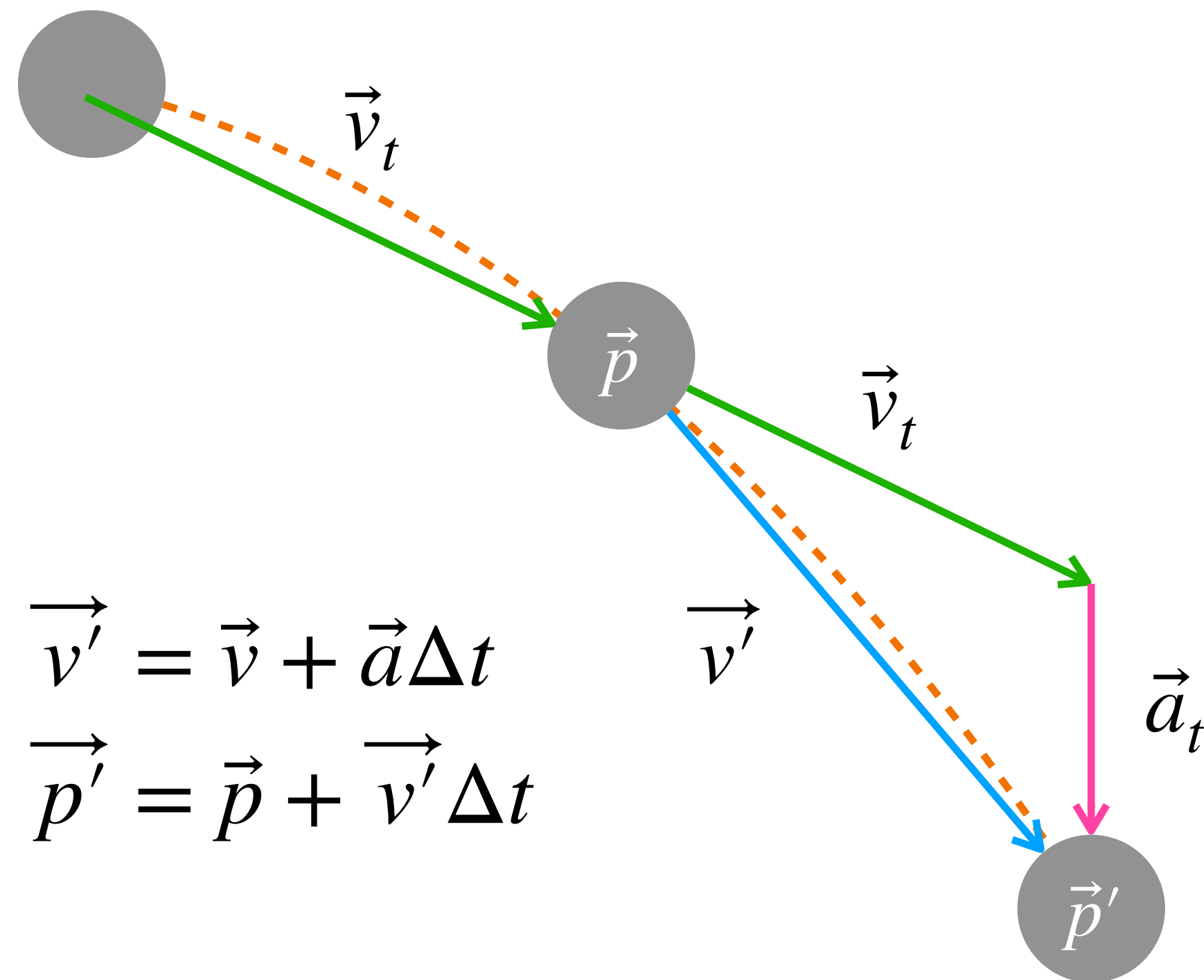


Mais preciso! Por isso, uma das opções mais comuns para implementação de objetos rígidos em jogos!

Método de Euler Semi-Implicito



Geometricamente, as curvas dos movimentos contínuos reais são aproximados por uma sequência de retas:



$$\vec{v}' = \vec{v} + \vec{a}\Delta t$$

$$\vec{p}' = \vec{p} + \vec{v}'\Delta t$$

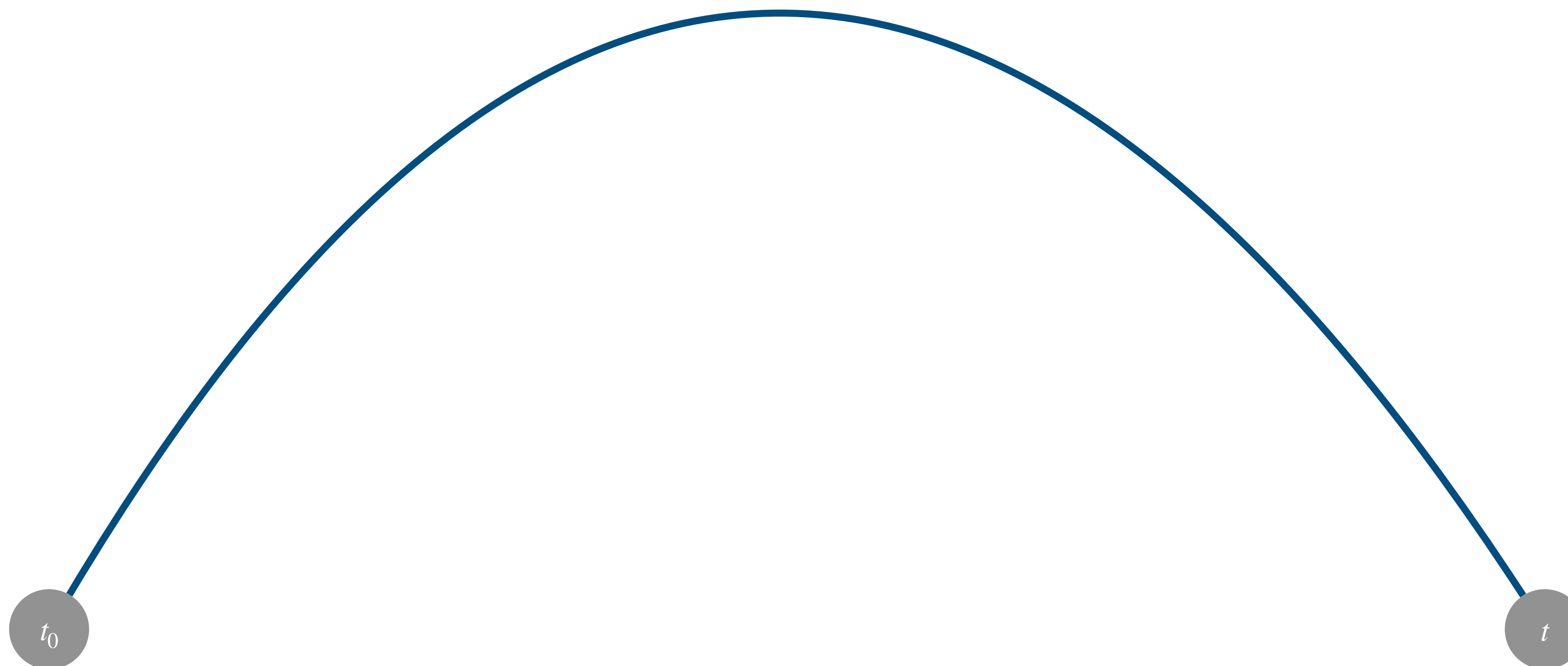
```
void Update(const float deltaTime) {  
    mVelocity += mAcceleration * dt;  
    mPosition += mVelocity * dt;  
    mAcceleration.Set(.0, .0);  
}
```

- ▶ O intervalo de tempo Δt define a magnitude da aceleração $\vec{a}_t$
- ▶ Quanto menor o Δt , melhor a aproximação do **movimento real**.

Impacto do tamanho do delta time



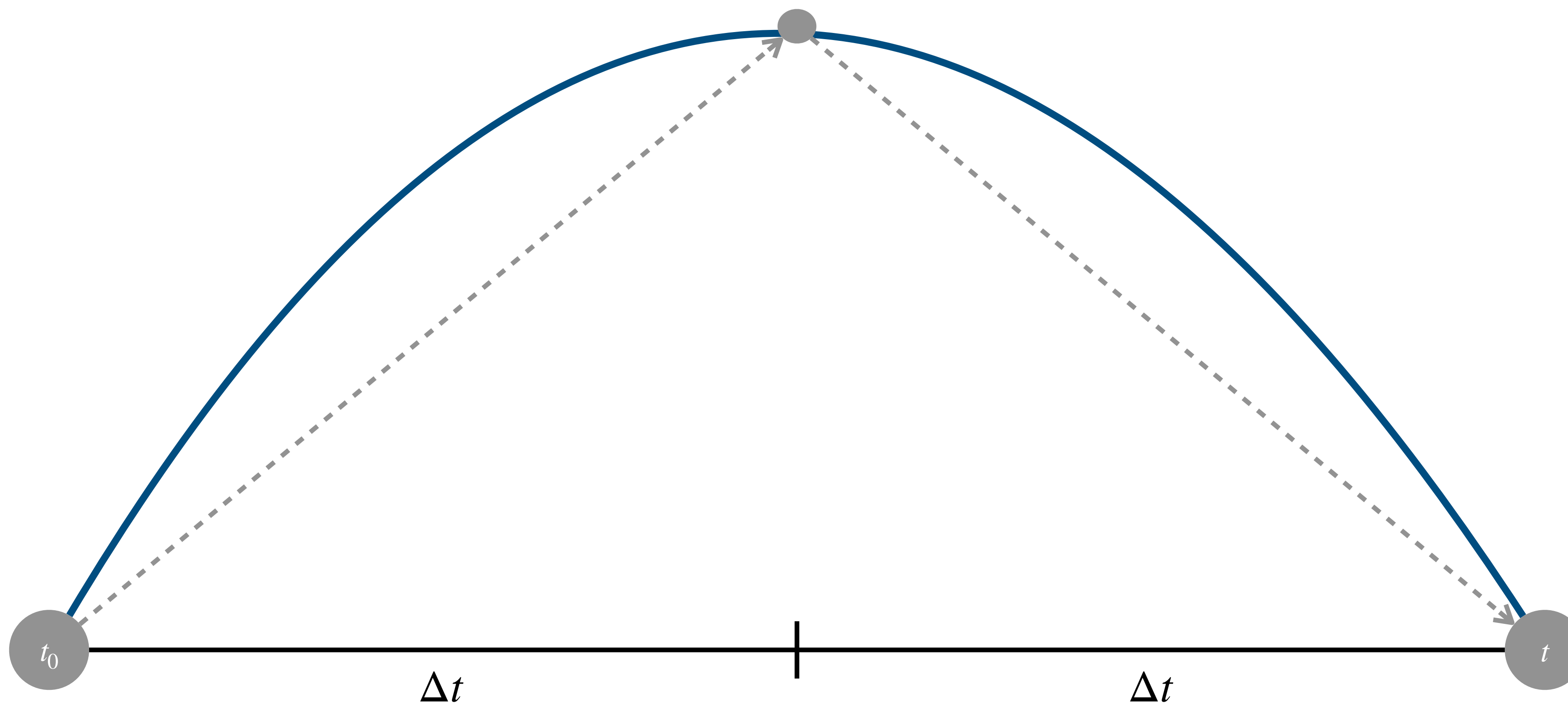
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



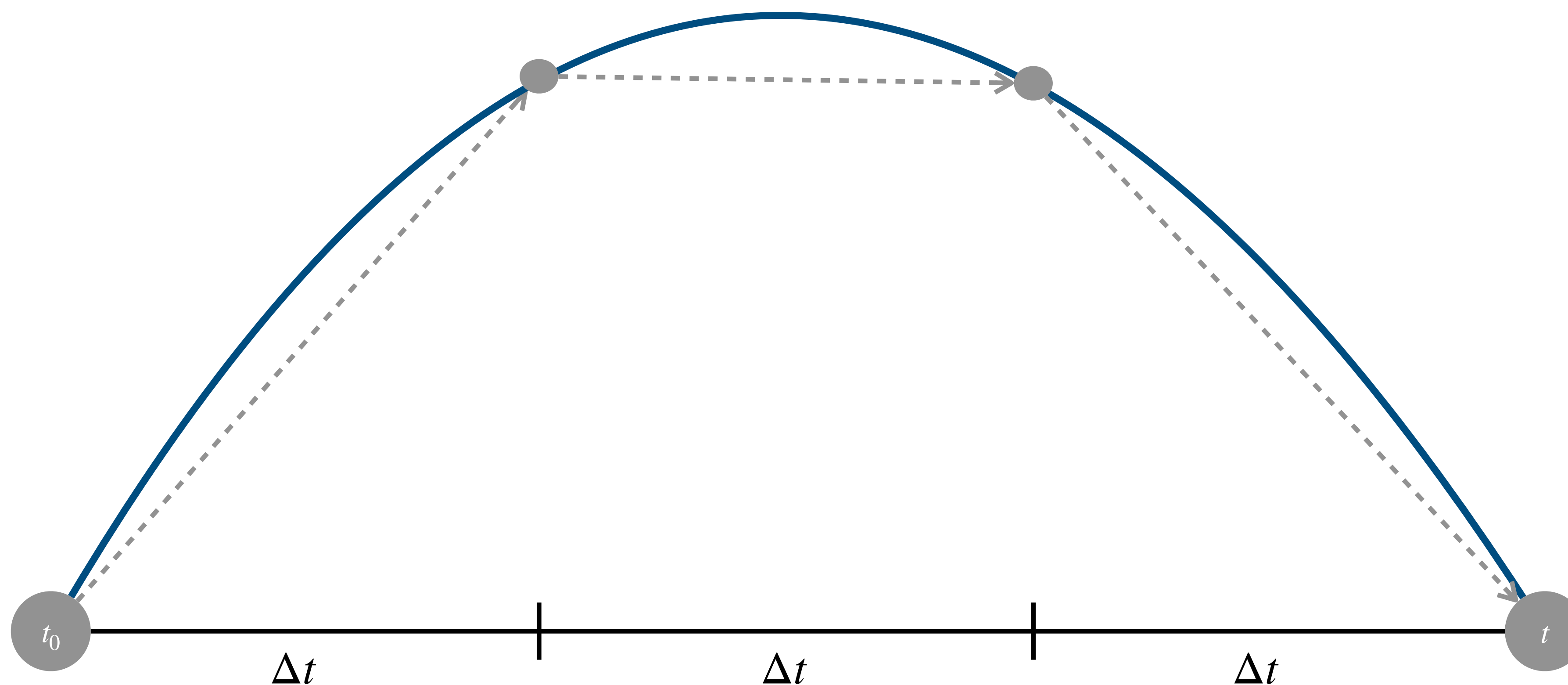
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



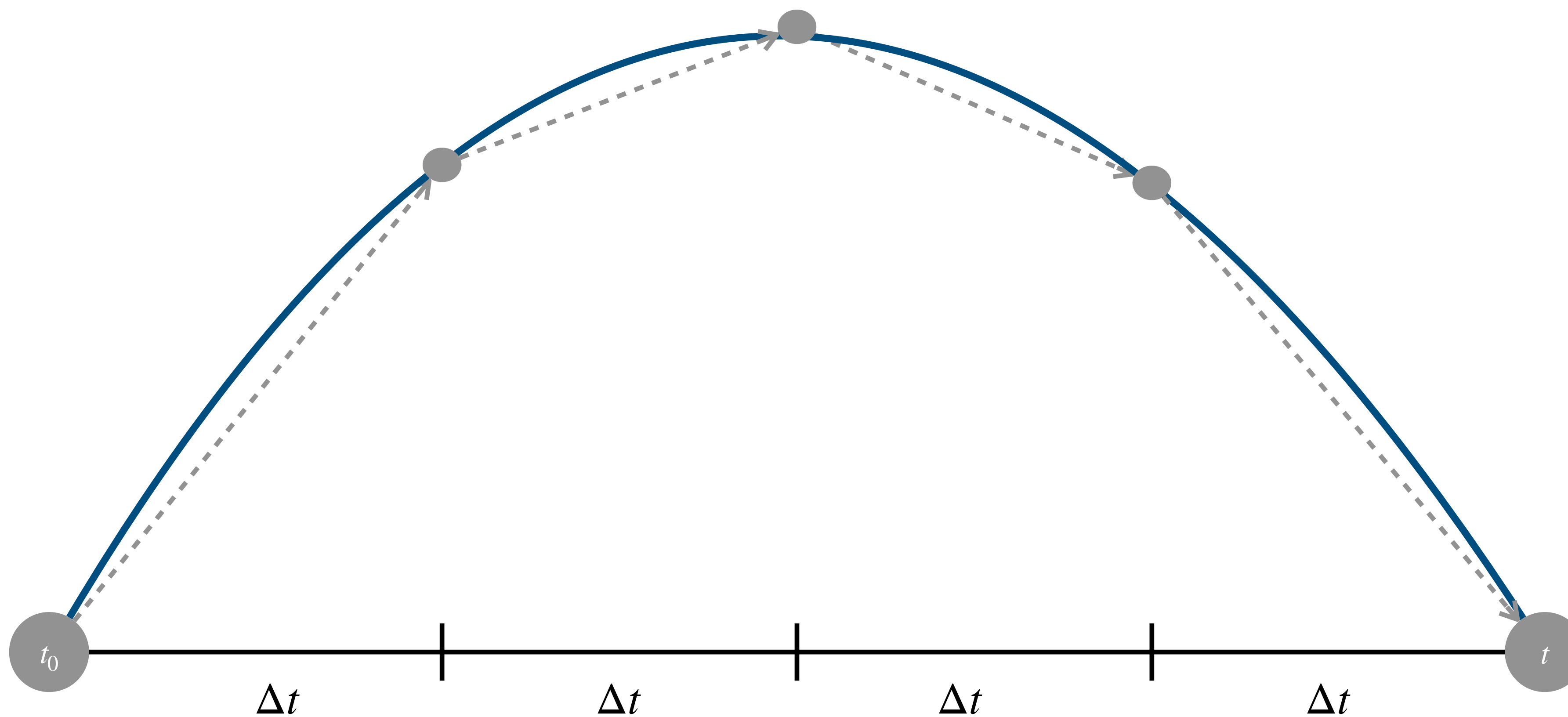
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



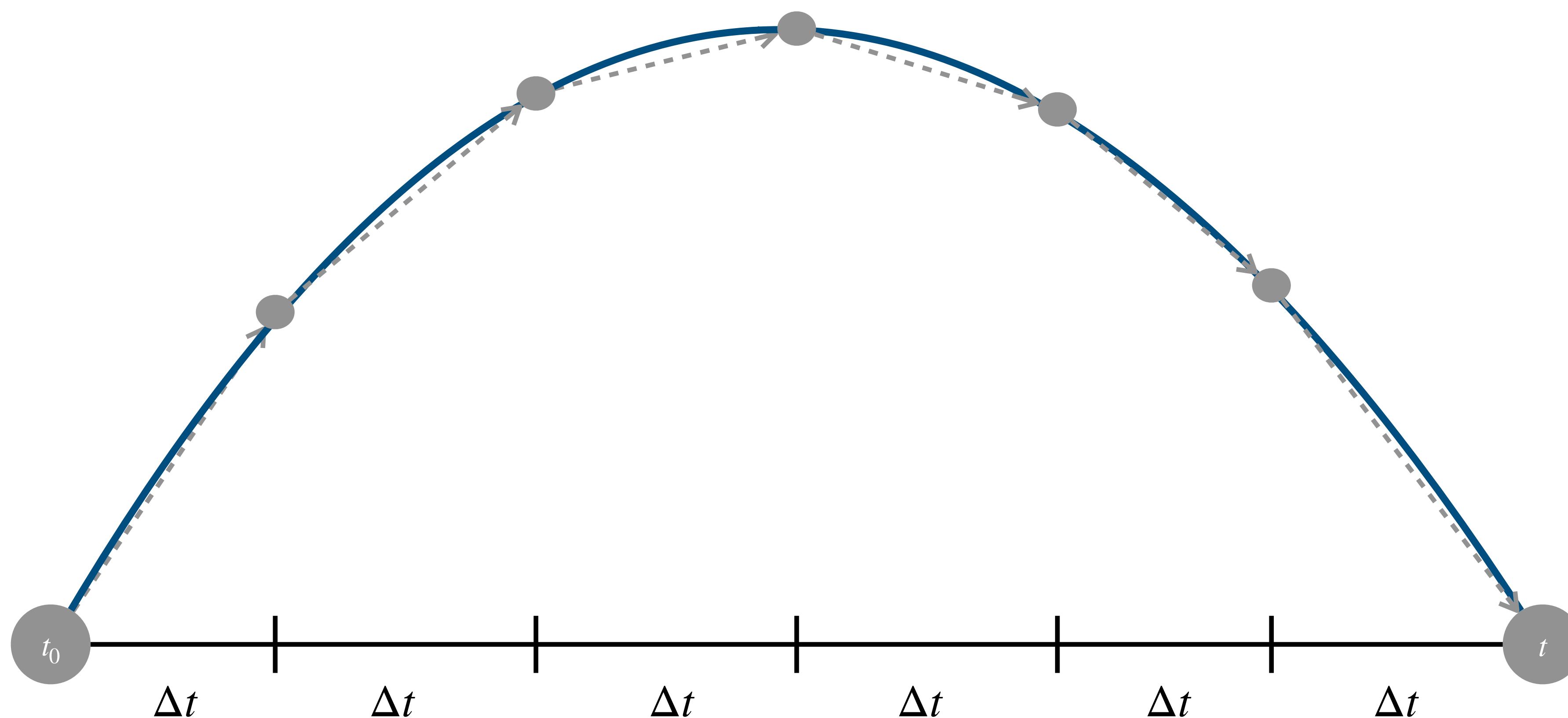
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



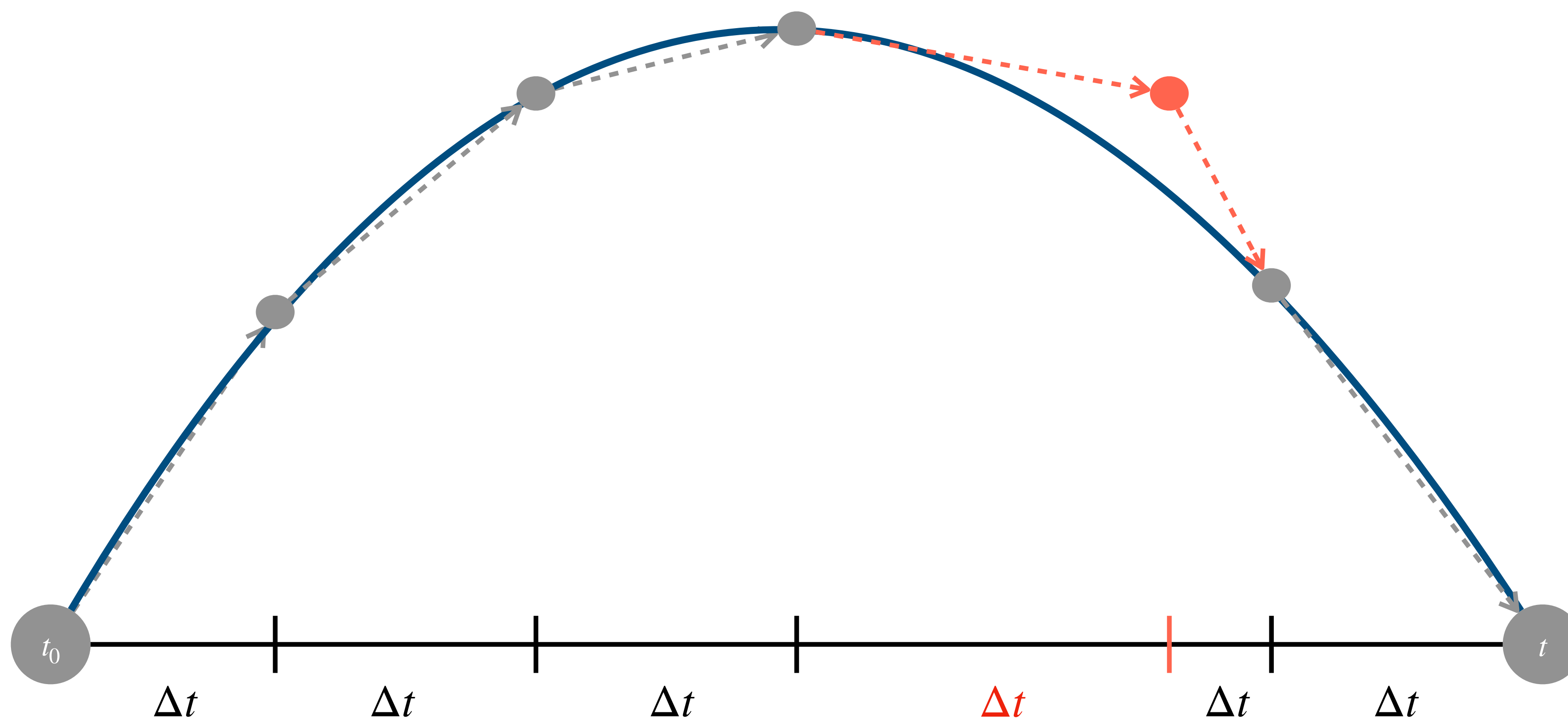
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto na variação do delta time



Como mencionados em aulas anteriores, se o delta time Δt for variável entre quadros, a simulação física pode ficar instável!



Implementando Objetos Rígidos



Uma boa forma de implementar Objetos Rígidos é utilizando um componente que pode ser adicionado à lista de componentes dos objetos que terão movimento do jogo:

```
class RigidBody : public Component
{
public:
    RigidBody(class Actor* owner, float mass);
    void ApplyForce(const Vector2 &force);
    void Update(float deltaTime) override;

private:

    // Physical properties
    float mMass;
    Vector2 mVelocity;
    Vector2 mAcceleration;
};
```

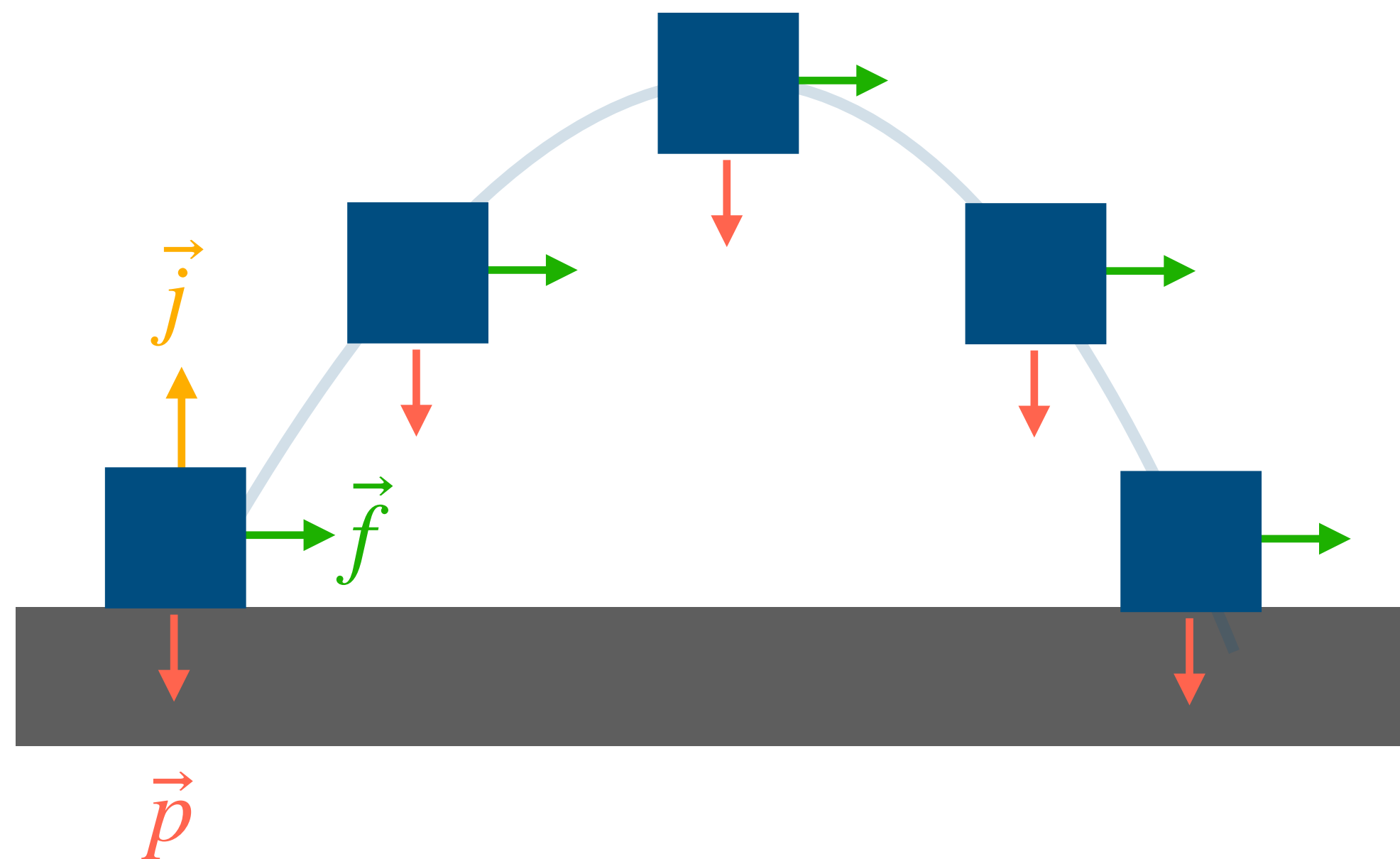
```
void Update(const float deltaTime) {
    mVelocity += mAcceleration * dt;
    mPosition += mVelocity * dt;
    mAcceleration.Set(.0, .0);
}
```

```
void ApplyForce(const Vector2 &force) {
    mAcceleration += force * (1.f/mMass);
}
```

Aceleração da Gravidade



Para implementar uma **Força Peso** nos objetos, basta aplicar uma força constante para baixo a cada atualização do componente Rigidbody:

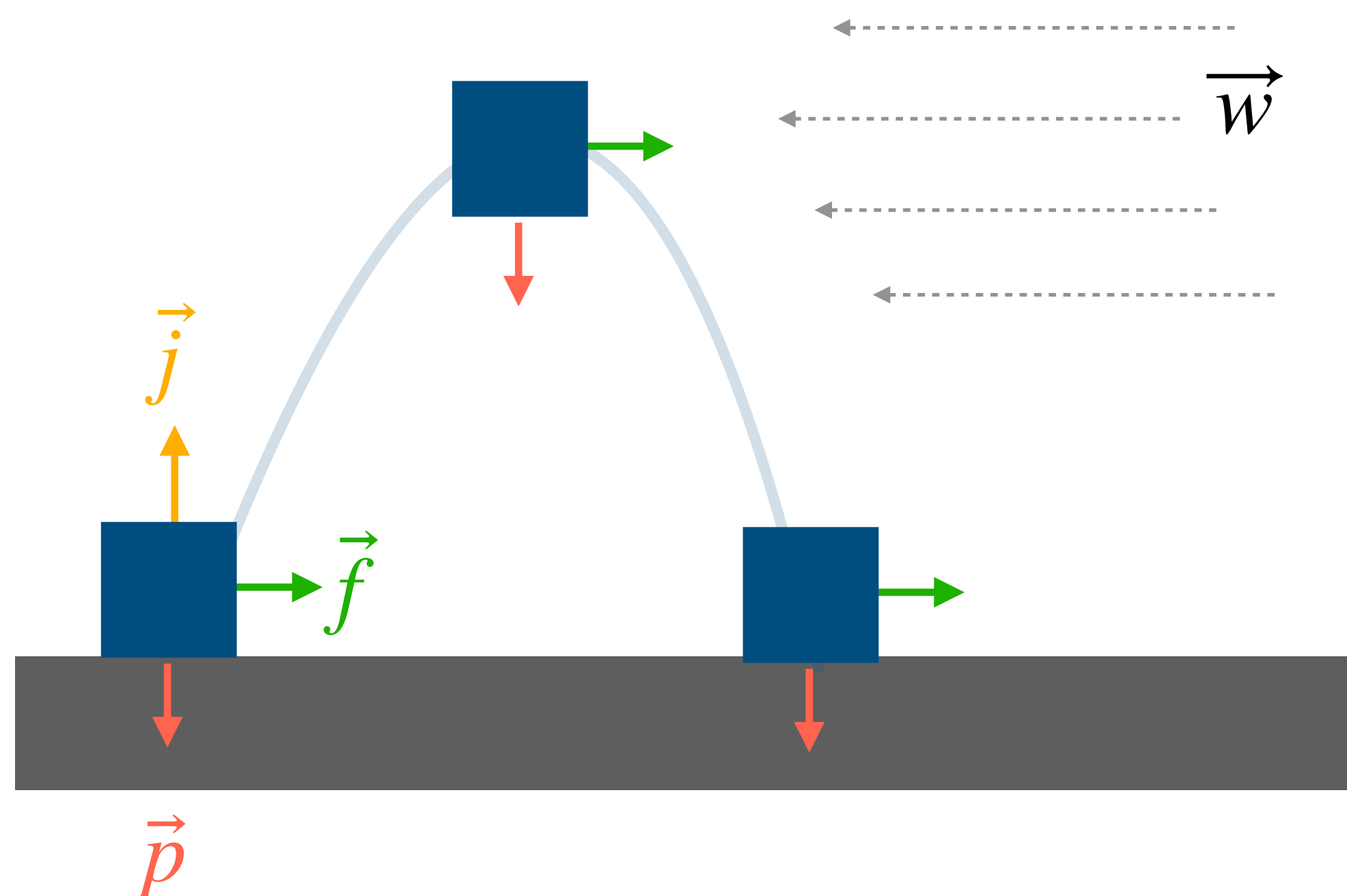


```
Vector2 g = Vector2(0.f, 9.8f);
```

```
Rigidbody::Update(float dt) {  
    ApplyForce(mMass * g);  
    mVelocity += mAcceleration * dt;  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```

```
Rigidbody::ApplyForce(Vector2 f) {  
    mAcceleration += f * 1f/mMass;  
}
```


De forma similar, para simular uma força causada por **vento**, que atua nos objetos em uma determinada direção, basta aplicar uma força constante $\vec{w}$ nessa direção:



```
Vector2 g = Vector2(0.f, 9.8f);
Vector2 w = Vector2(-10.0f, 0.0f);

RigidBody::Update(float dt) {
    ApplyForce(mMass * g);
    ApplyForce(w);

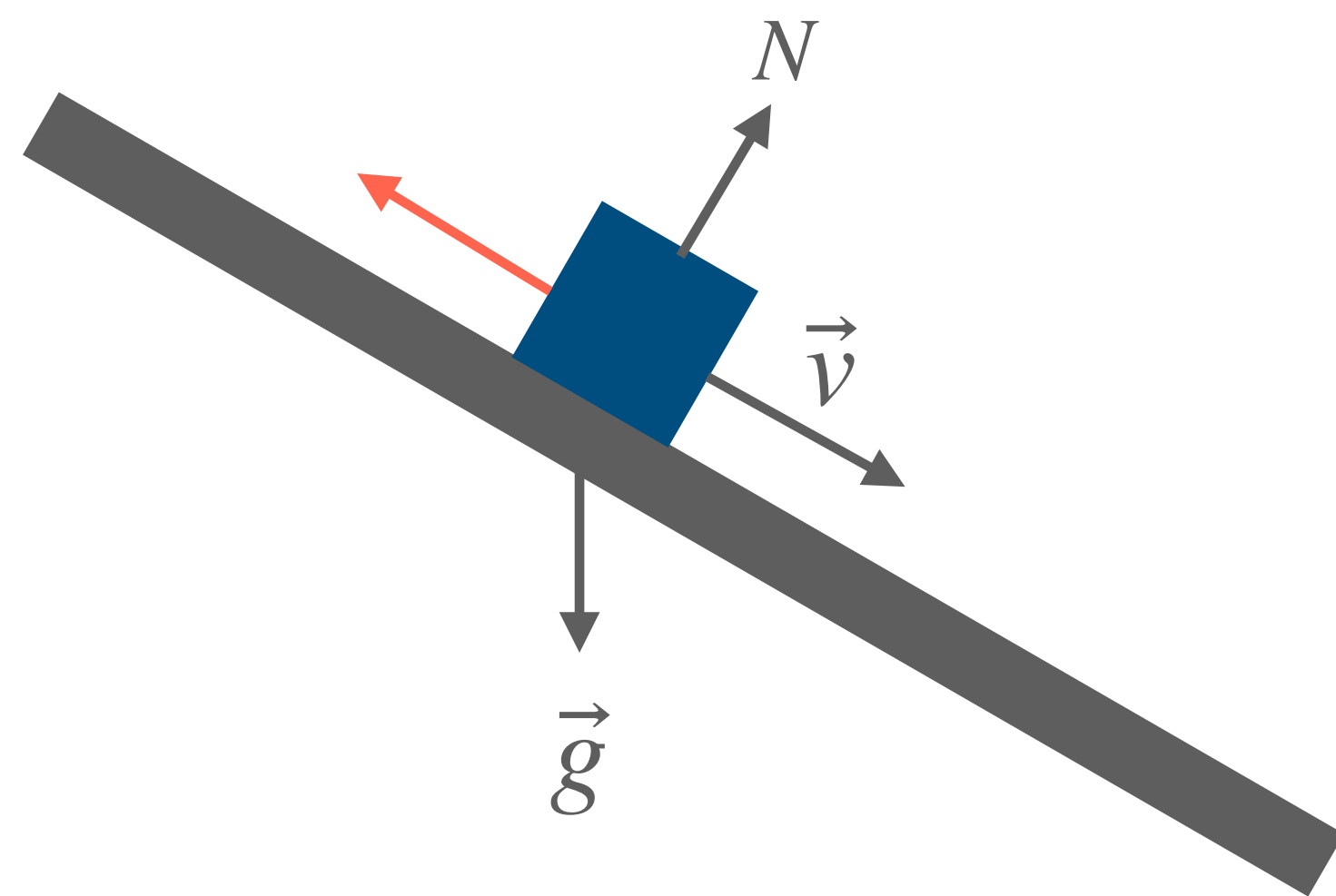
    mVelocity += mAcceleration * dt;
    mPosition += position * dt;
    mAcceleration.Set(0f, 0f);
}
```

Atrito



Para aplicar uma **Força de Atrito**, podemos calcular o inverso da velocidade normalizada e multiplicar por um coeficiente de atrito μ e a magnitude do vetor normal à superfície N :

$$\vec{F}_a = -1\mu ||N||\hat{v}$$



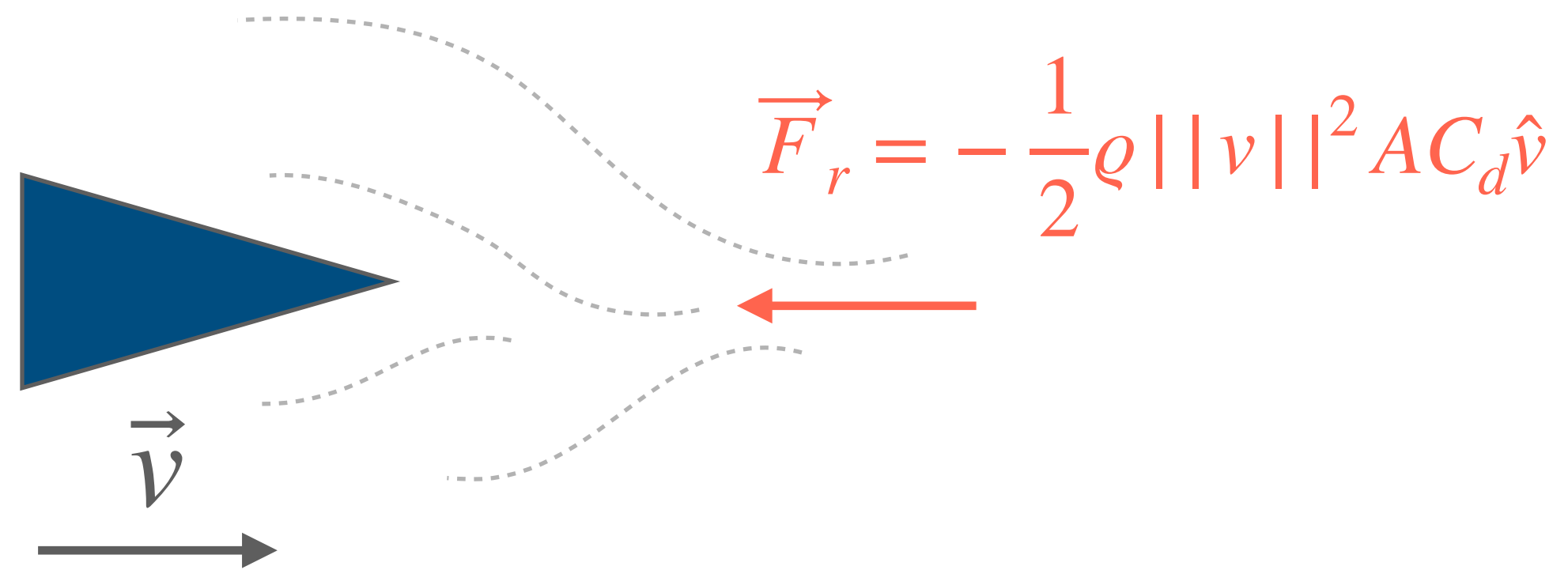
- ▶ μ : coeficiente de atrito
- ▶ N : é a força normal

```
RigidBody::ApplyFriction(float u, Vector2 N) {  
    if(mVelocity.Length() <= a)  
        return;  
  
    float frictionMag = u * N.Length();  
    Vector2 friction = (-1) * mVelocity;  
    friction.Normalize();  
    friction *= frictionMag;  
  
    ApplyForce(friction);  
}
```

Resistência do Meio



A mesma ideia se aplica para parar um objeto que não está em contato com uma superfície, mas sobre **resistência do meio**, como do ar ou de um fluido.



- ▶ ρ : densidade do meio
- ▶ $||v||^2$: comprimento do vetor velocidade
- ▶ A : área frontal do objeto
- ▶ C_d : coeficiente de resistência
- ▶ $\hat{v}$: direção do vetor velocidade

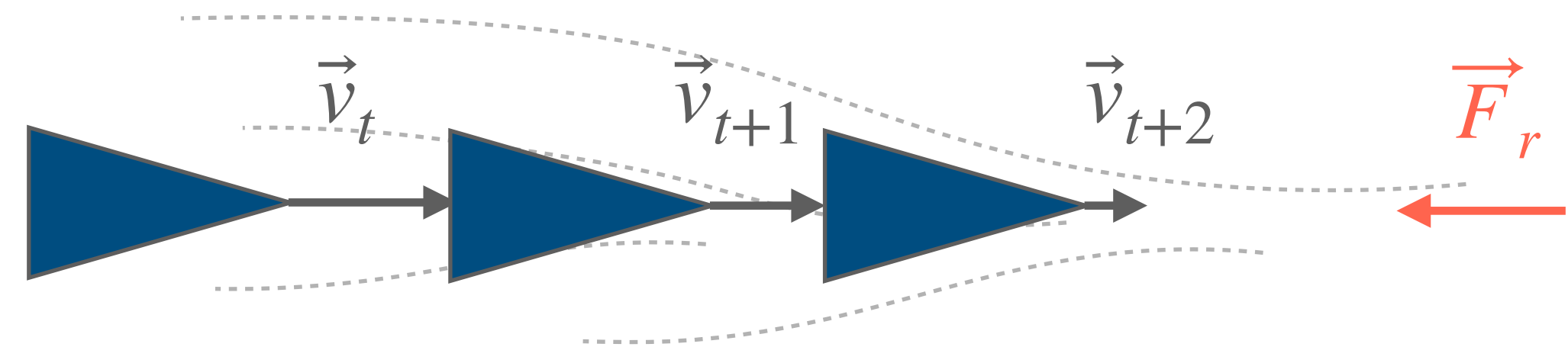
```
RigidBody::ApplyDrag(float rho) {  
    if(mVelocity.Length() <= MIN_VEL)  
        return;  
  
    float vLen = mVelocity.Length();  
    float dragLen = rho * vLen * vLen;  
  
    Vector2 drag = (-1) * mVelocity;  
    drag.Normalize();  
    drag *= dragLen;  
  
    ApplyForce(drag);  
}
```


Parando por Completo



Note que a aceleração é zerada a cada quadro, mas e a **velocidade não**, ou seja, o objeto só irá parar por completo quando a aceleração cancelar perfeitamente a velocidade atual:

```
RigidBody::Update(float dt) {  
    ApplyDrag(0.1f);  
    mVelocity += mAcceleration * dt;  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```



Problemas:

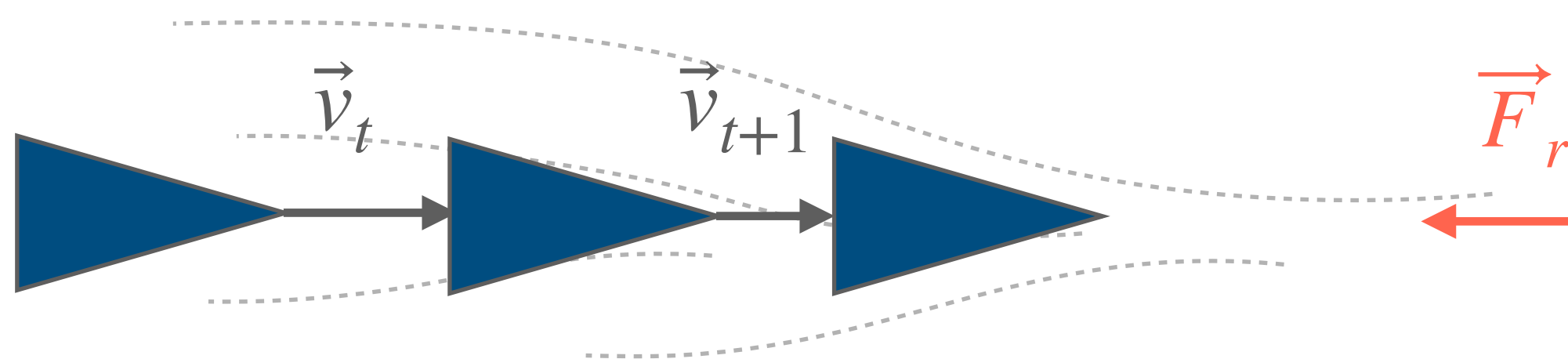
- ▶ $\vec{v} = (0,0)$ é extremamente improvável devido às multiplicações por dt e pela natureza de ponto flutuante das forças
- ▶ O objeto vai chegar a uma velocidade escalar muito baixa (ex. 0.001), mas nunca irá parar por completo

Parando por Completo



Note que a aceleração é zerada a cada quadro, mas e a **velocidade não**, ou seja, o objeto só irá parar por completo quando a aceleração cancelar perfeitamente a velocidade atual:

```
RigidBody::Update(float dt) {  
    ApplyDrag(0.1f);  
    mVelocity += mAcceleration * dt;  
  
    // Verificar velocidade mínima  
    if(mVelocity.Length() < MIN_VEL) {  
        mVelocity.Set(.0f, .0f);  
    }  
  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```



Solução:

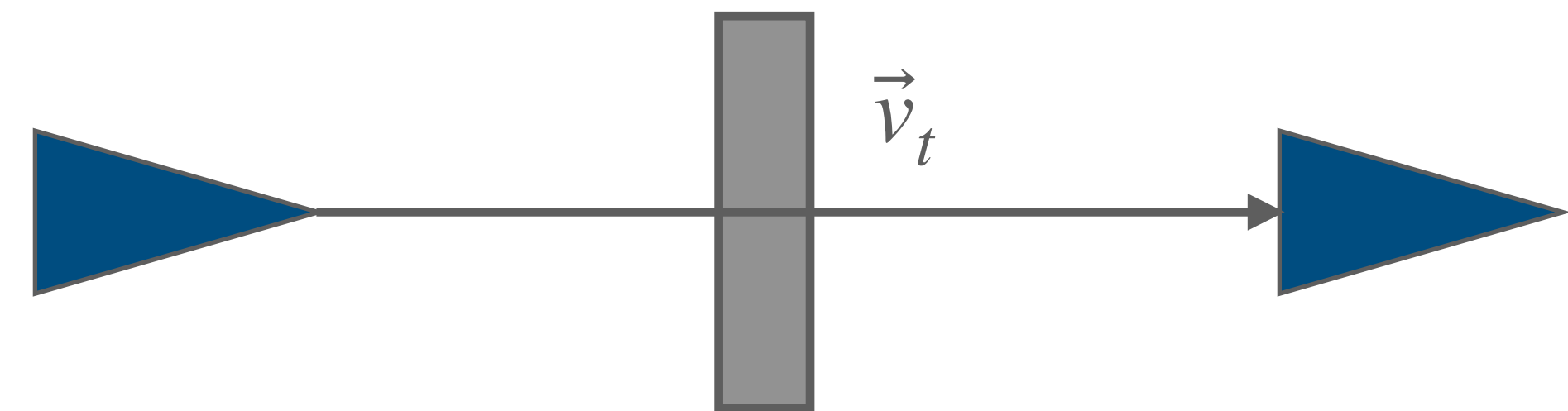
- ▶ Definir um limiar de velocidade escalar mínima `MIN_VEL`
- ▶ Se a velocidade escalar for menor que esse limiar, force ela a ser exatamente zero

Velocidade Máxima



Também é importante definir um limiar de **velocidade máxima**, para tornar a simulação mais controlada e evitar comportamentos inesperados (ex. atravessar paredes)

```
RigidBody::Update(float dt) {  
    mVelocity += mAcceleration * dt;  
    // Verificar velocidade mínima  
    ...  
    // Verificar velocidade máxima  
    if(mVelocity.Length() > MAX_VEL) {  
        mVelocity.Normalize();  
        mVelocity *= MAX_VEL;  
    }  
  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```



Solução:

- ▶ Definir um limiar de velocidade escalar máxima MAX_VEL
- ▶ Se a velocidade escalar for maior que esse limiar, force ela a ser exatamente MAX_VEL

Próxima aula



A8: Física II – Detecção de Colisão

- ▶ Geometrias de colisão
 - ▶ Falsos Positivos
- ▶ Detecção de colisão
 - ▶ Circunferência vs. Circunferência
 - ▶ AABB vs. AABB
- ▶ Resolução de Colisão
 - ▶ AABB vs. AABB